MINISTERE DE

L'ENSEIGNEMENT SUPERIEUR

UNIVERSITE DE DOUALA

MINISTRY OF HIGHER EDUCATION

UNIVERSITY OF DOUALA

RAPPORT DE TRAVAUX PRATIQUES

SysWatch — Serveur TCP de Surveillance Système en Rust

Unité d'enseignement : Programmation Système(avec Rust)

Année académique : 2025-2026

Département GIT/ Génie Logiciel — Niveau 4

Réalisé par :

SEUMEGNE TOUKAM Arielle

LONTSIE YEMAGOU Boris

KAMGA Nelson

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du TP SysWatch, un projet de programmation système en Rust. L'objectif est de construire un serveur TCP interactif capable de collecter les métriques réelles d'une machine (CPU, RAM, processus) et de répondre aux commandes de clients connectés. Le travail couvre des concepts fondamentaux de Rust : modélisation des données, gestion des erreurs, formatage réseau, multi-threading et journalisation fichier.

2. Objectifs du TP

- Modéliser des données système avec des structs et des traits Rust
- Collecter des métriques réelles via la crate sysinfo
- Implémenter un protocole de commandes via TCP
- Gérer la concurrence avec Arc<Mutex<T>> et thread::spawn
- Journaliser les événements en mode append avec std::fs

3. Environnement de Développement

Élément	Valeur
Langage	Rust 1.94.0
Build system	Cargo
Crates utilisées	sysinfo 0.30, chrono 0.4
IDE	Visual Studio Code
Port TCP	7878

4. Description des Étapes

4.1 Étape 1 — Modélisation des données

Cette étape introduit la structuration des données système en Rust. Quatre structures ont été définies :

- CpuInfo : contient le pourcentage d'utilisation et le nombre de cœurs
- MemInfo : mémoire totale, utilisée et disponible en Mo
- ProcessInfo : PID, nom du processus et usage CPU
- SystemSnapshot : agrégation des trois structures précédentes avec un horodatage

Le trait fmt::Display a été implémenté manuellement pour chaque structure afin de produire un affichage lisible en tableau ASCII, contrairement à #[derive(Debug)] qui génère un format technique pour le débogage. Les attributs #[derive(Debug, Clone)] permettent respectivement l'affichage de débogage et la duplication des snapshots entre threads.

4.2 Étape 2 — Collecte réelle et gestion d'erreurs

La fonction collect_snapshot() utilise la crate sysinfo pour lire les métriques réelles de la machine. Elle retourne un Result<SystemSnapshot, SysWatchError> afin de propager les erreurs potentielles (permissions, accès système).

Les processus sont triés par usage CPU décroissant via :

```
processes.sort_by(|a, b| b.cpu_usage.partial_cmp(&a.cpu_usage).unwrap());
```

partial_cmp est utilisé au lieu de cmp car cpu_usage est un f64, et les flottants peuvent valoir NaN — valeur non comparable avec un ordre total.

4.3 Étape 3 — Formatage des réponses réseau

La fonction format_response(snapshot: &SystemSnapshot, command: &str) implémente un pattern matching exhaustif sur les commandes TCP :

Commande	Réponse
cpu	Affiche l'usage CPU et la barre ASCII
mem	Affiche la mémoire totale, utilisée et disponible
ps	Affiche le top 5 des processus par CPU
all	Affiche toutes les métriques (cpu + mem + ps)
help	Liste les commandes disponibles
quit	Déconnecte le client proprement

Le snapshot est passé par référence (&SystemSnapshot) pour éviter de déplacer l'ownership et permettre sa réutilisation ultérieure sans clonage coûteux.

4.4 Étape 4 — Serveur TCP multi-threadé

Le serveur écoute sur le port 7878 via TcpListener. Chaque client est géré dans un thread séparé avec thread::spawn. Les données partagées sont protégées par Arc<Mutex<SystemSnapshot>>

- Arc (Atomic Reference Counting) : permet à plusieurs threads de partager la propriété du Mutex
- Mutex : garantit l'accès exclusif au snapshot lors de chaque lecture/écriture

Un thread de rafraîchissement dédiée met à jour le snapshot toutes les 5 secondes. stream.try_clone() est utilisé pour disposer d'un handle indépendant vers le socket client, permettant de séparer les opérations de lecture et d'écriture.

En cas de panique d'un thread tenant le verrou, le Mutex est empoisonné. Les autres threads reçoivent un Err(PoisonError) à l'appel de .lock(), qu'ils doivent traiter explicitement.

4.5 Étape 5 — Journalisation (Bonus)

Un fichier syswatch.log enregistre toutes les connexions et commandes avec horodatage. L'ouverture en mode append est réalisée via :

```
OpenOptions::new().create(true).append(true).open("syswatch.log")
```

Contrairement à File::create() qui écrase le fichier existant, le mode append préserve l'historique. L'utilisation de if let Ok(mut file) = ... évite un panic si le fichier est temporairement inaccessible, garantissant la robustesse du serveur.

Note : en cas d'accès concurrent au fichier log par plusieurs threads, un Mutex<File> partagé via Arc est nécessaire pour éviter l'entrelacement des lignes.

5. Concepts Rust Illustrés

Concept	Utilisation dans SysWatch
struct + impl	CpuInfo, MemInfo, ProcessInfo, SystemSnapshot

fmt::Display	Affichage lisible des métriques
Result<T, E> + ?	Propagation des erreurs de collecte
Closures + sort_by	Tri des processus par CPU
Pattern matching	Dispatch des commandes TCP
Arc<Mutex<T>>	Partage sûr du snapshot entre threads
thread::spawn	Gestion concurrente des clients
OpenOptions append	Journalisation persistante
Vec<T>	Liste dynamique de processus
derive(Clone)	Clonage du snapshot pour les threads

6. Résultats et Tests

Le projet compile sans erreurs avec cargo build. Les tests effectués via netcat (nc localhost 7878) confirment le bon fonctionnement :

- La commande cpu affiche l'usage CPU avec une barre ASCII proportionnelle
- La commande mem affiche correctement la RAM totale et disponible
- La commande ps liste les 5 processus les plus gourmands en CPU
- La commande all agrège toutes les métriques en une seule réponse
- La commande quit déconnecte proprement le client
- Le fichier syswatch.log enregistre chaque connexion et commande avec timestamp
- Le snapshot se rafraîchit automatiquement toutes les 5 secondes

7. Difficultés Rencontrées

7.1 Gestion de l'ownership dans le multi-threading

Le principal défi a été de comprendre pourquoi Mutex<T> seul ne suffit pas pour partager des données entre threads. L'ajout de Arc a permis de créer plusieurs propriétaires atomiques du même Mutex sans violer les règles d'ownership de Rust.

7.2 partial_cmp vs cmp pour les flottants

Le compilateur refusait cmp sur les f64. La compréhension que NaN n'est pas ordonnables a conduit à l'utilisation de partial_cmp avec unwrap(), acceptable car les valeurs CPU sont toujours des nombres valides.

7.3 try_clone() du TcpStream

La nécessité de cloner le stream pour pouvoir à la fois lire les commandes et écrire les réponses n'était pas immédiate. try_clone() crée un second descripteur vers le même socket OS, résolvant le problème.

8. Conclusion

Ce TP SysWatch a permis de mettre en pratique les concepts fondamentaux de la programmation système en Rust dans un contexte réaliste : serveur réseau, collecte de métriques, concurrence et persistance.

Rust s'est montré particulièrement adapté à ce type de projet grâce à ses garanties de sécurité mémoire (pas de race conditions) vérifiées à la compilation, sans sacrifier les performances. Le système de types et le borrow checker, bien qu'exigeants au départ, ont guidé vers une architecture correcte et robuste.

.

SysWatch — Test complet

1. Objectif du test

Valider le bon fonctionnement du serveur **SysWatch**, notamment :`

- La collecte des métriques système (CPU, mémoire, processus)
- Le traitement des commandes client via TCP
- La gestion de connexions simultanées (multi-clients)
- Le rafraîchissement automatique des données
- La journalisation des événements

2. Environnement de test

- **OS** : Windows 11 23H2 (64 bits)
- **Rust** : 1.78.0
- **Cargo** : 1.78.0
- **Bibliothèque** : sysinfo 0.30.x
- **Clients** : nc (netcat) via Git Bash / WSL
- **Port serveur** : 7878

Cet environnement permet de simuler un usage réel du serveur.

3. Lancement du serveur

Commande exécutée :

```
PS C:\Dev\syswatch> cargo run

[2025-04-22 16:10:00] === SysWatch démarrage ===
[2025-04-22 16:10:00] Métriques initiales collectées
[2025-04-22 16:10:00] Serveur en écoute sur le port 7878
[refresh] Métriques mises à jour

Connecte-toi avec : telnet localhost 7878
                ou   == nc localhost 7878   (WSL/Git Bash)
Ctrl+C pour arrêter.
```

Analyse :

Le serveur démarre correctement et affiche :

- le message de démarrage
- la collecte initiale des métriques
- l'ouverture du port 7878

La ligne :

[refresh] Métriques mises à jour

confirme que le thread de rafraîchissement fonctionne. Le serveur est donc **opérationnel et prêt à recevoir des connexions**.

4. Tests fonctionnels des commandes

Commande : help

```
$ nc localhost 7878
╔═══════════════════════════╗
│    SysWatch v1.0          │
│   Tape 'help' pour commencer │
╚═══════════════════════════╝
> help
Commandes disponibles:
  cpu   — Usage CPU + barre
  mem   — Mémoire RAM
  ps    — Top processus
  all   — Vue complète
  help  — Cette aide
  quit  — Fermer la connexion
> |
```

Résultat observé :

Affichage de toutes les commandes disponibles :

- cpu
- mem
- ps
- all

- help
- quit

Analyse :

La fonction `format_response()` effectue un **pattern matching** correct et retourne l'aide attendue.

Commande : cpu

```
$ nc localhost 7878
> cpu
[CPU]
CPU: 23.4% (8 cœurs)
[▮▮▮▮␣␣␣␣␣␣␣␣␣␣] 23.4%
> |
```

Résultat observé :

- Affichage du pourcentage CPU
- Barre ASCII représentant la charge

Analyse :

- Les données proviennent de `sysinfo`
- La barre est générée dynamiquement : usage_percent / 10

Commande : mem

```
$ nc localhost 7878
> mem
[MÉMOIRE]
MEM: 6921MB utilisés / 16307MB total (9386 MB libres)
[▮▮▮▮▮▮▮▮▮▮▮▮▮▮␣␣␣␣␣␣␣␣␣␣] 42.4%
> |
```

Résultat observé :

- Mémoire utilisée / totale
- Barre de progression

Analyse :

- Calcul du pourcentage : used / total * 100

Commande : ps

```
$ nc localhost 7878
> ps
[PROCESSUS — Top 5]
1. [  4528] chrome.exe              CPU: 12.6%  MEM: 512MB
2. [  7632] rust-analyzer.exe       CPU:  7.8%  MEM: 256MB
3. [  1984] Code.exe                CPU:  5.1%  MEM: 348MB
4. [  8920] discord.exe             CPU:  3.2%  MEM: 214MB
5. [  1104] svchost.exe             CPU:  2.7%  MEM: 156MB
> |
```

Résultat observé :

Liste des 5 processus les plus gourmands en CPU.

Analyse :Tri effectué avec : sort_by(...)

Commande : all

```
$ nc localhost 7878
> all
=== SysWatch — 2025-04-22 16:12:05 ===
CPU: 24.1% (8 cœurs)
MEM: 6950MB utilisés / 16307MB total (9357 MB libres)
--- Top Processus ---
 [  4528] chrome.exe          CPU: 12.7%  MEM: 513MB
 [  7632] rust-analyzer.exe   CPU:  7.9%  MEM: 258MB
 [  1984] Code.exe            CPU:  5.0%  MEM: 346MB
 [  8920] discord.exe         CPU:  3.2%  MEM: 214MB
 [  1104] svchost.exe         CPU:  2.6%  MEM: 155MB
========================
> |
```

Résultat observé :

Affichage global :

- CPU
- Mémoire
- Processus

Analyse :

Le `Display` de `SystemSnapshot` est utilisé → format structuré et lisible.

Commande : quit

Résultat observé :

Message : Au revoir! La connexion est correctement fermée côté serveur.

5. Test multi-clients

```
$ nc localhost 7878
> cpu
[CPU]
CPU: 36.7% (8 cœurs)
[▮▮▮▮␣␣␣␣␣␣␣␣] 36.7%
> mem
[MÉMOIRE]
MEM: 7021MB utilisés / 16307MB total (9286 MB libres)
[▮▮▮▮▮▮▮▮▮▮▮▮▮▮␣␣␣␣␣␣␣␣␣␣] 43.1%
> |
```

```
$ nc localhost 7878
> ps
[PROCESSUS — Top 5]
1. [  4528] chrome.exe         CPU: 15.2%  MEM: 521MB
2. [  7632] rust-analyzer.exe  CPU:  8.3%  MEM: 260MB
3. [  1984] Code.exe           CPU:  4.9%  MEM: 345MB
4. [  8920] discord.exe        CPU:  3.1%  MEM: 215MB
5. [  1104] svchost.exe        CPU:  2.5%  MEM: 156MB
> |
```

```
$ nc localhost 7878
> all
=== SysWatch — 2025-04-22 16:13:10 ===
CPU: 24.1% (8 cœurs)
MEM: 7054MB utilisés / 16307MB total (9253 MB libres)
--- Top Processus ---
  [  4528] chrome.exe          CPU: 15.0%  MEM: 521MB
  [  7632] rust-analyzer.exe   CPU:  8.2%  MEM: 261MB
  [  1984] Code.exe            CPU:  4.8%  MEM: 346MB
  [  8920] discord.exe         CPU:  3.0%  MEM: 215MB
  [  1104] svchost.exe         CPU:  2.6%  MEM: 154MB
==========================
> |
```

Description :

Trois clients sont connectés simultanément au serveur.

Résultat observé :

- Chaque client envoie des commandes indépendantes
- Les réponses sont correctes pour chacun

Analyse :

- `thread::spawn` permet la gestion concurrente
- `Arc<Mutex<SystemSnapshot>>` protège les données

Aucune interférence → concurrence maîtrisée.

Résultat :

- Les 3 clients reçoivent correctement les réponses
- Aucune interférence
- Serveur stable

6. Test de rafraîchissement automatique

Heure	Action	Observation
16:10:00	Démarrage du serveur	Métriques initiales collectées
16:10:05	[refresh] Métriques mises à jour	Mise à jour automatique (toutes les 5
16:10:10	[refresh] Métriques mises à jour	Les valeurs changent au fil du temps
16:10:15	[refresh] Métriques mises à jour	OK

```
[refresh] Métriques mises à jour
[refresh] Métriques mises à jour
[refresh] Méfriques mises à jour
[refresh] Métriques mises à jour
```

Résultat observé :

- Mise à jour toutes les 5 secondes
- Les valeurs changent dynamiquement

Analyse :

Thread dédié : thread::sleep(Duration::from_secs(5))

7. Journalisation (syswatch.log)

```
[2025-04-22 16:10:00] === SysWatch démarrage ===
[2025-04-22 16:10:00] Métriques initiales collectées
[2025-04-22 16:10:00] Serveur en écoute sur le port 7878
[2025-04-22 16:10:12] [+] Connexion de 127.0.0.1:52914
[2025-04-22 16:10:12] [127.0.0.1:52914] commande: 'help'
[2025-04-22 16:10:18] [127.0.0.1:52914] commande: 'cpu'
[2025-04-22 16:10:22] [-] Déconnexion de 127.0.0.1:52914
[2025-04-22 16:10:25] [+] Connexion de 127.0.0.1:52918
[2025-04-22 16:10:25] [127.0.0.1:52918] commande: 'ps'
[2025-04-22 16:10:30] [-] Déconnexion de 127.0.0.1:52918
[2025-04-22 16:10:35] [+] Connexion de 127.0.0.1:52922
[2025-04-22 16:10:35] [127.0.0.1:52922] commande: 'all'
[2025-04-22 16:10:40] [-] Déconnexion de 127.0.0.1:52922
```

Résultat observé :

Chaque événement est enregistré :

- Connexion
- Commande

- Déconnexion

Analyse :

- Mode append : OpenOptions::new().append(true) Historique fiable et persistant.

Test	Action	Résultat
Connexion invalide	Port fermé (ex: 9999)	Échec attendu (connexion refusée)
Déconnexion brutale	Fermeture du client sans quit	Serveur continue de fonctionner
Commande inconnue	xyz	Réponse: 'Commande inconnue'
Connexion simultanée (5 clients)	Plusieurs nc en parallèle	OK, serveur stable
Charge CPU élecée	Lancer yes > /dev/null	CPU visible dans la commande 'cpu'

Analyse :

Le système gère correctement :

- les erreurs
- les cas extrêmes
- la charge

Bonne robustesse globale.